

API Security Risk Analysis Report

Future Interns – Cyber Security Internship

Task 3 – Read-Only API Security Risk Assessment

Author: Jeremiah Olatubosun

Executive Summary

This report documents a structured, ethical, and read-only security evaluation of a publicly accessible API platform. The objective of the assessment was to identify common API security weaknesses that may expose organizations to risks such as unauthorized access, excessive data disclosure, automated scraping, and service abuse. All testing activities were conducted using non-intrusive inspection techniques only, ensuring no disruption or exploitation occurred. The findings and recommendations presented in this report mirror the type of deliverable provided by professional security consultancies.

Scope & Ethical Boundaries

The assessment was limited strictly to publicly available demo endpoints. Only safe HTTP GET requests were issued. No exploitation, bypass attempts, or stress testing were performed. The analysis focused exclusively on observation, configuration review, and risk identification.

Tools Utilized

- Postman – endpoint request execution and response inspection
- Browser Developer Tools – header, network, and timing analysis
- Microsoft Word – professional documentation and reporting

Methodology

The API documentation was first reviewed to understand available endpoints. Requests were then sent using Postman to observe how the API responds without authentication. Headers, response bodies, and server behaviors were examined to determine whether security controls such as authentication, rate limiting, and data minimization were implemented. Risks were categorized according to their potential business impact and industry best practices, referencing common weaknesses described in the OWASP API Security Top 10.

Endpoint Analysis – /posts

The /posts endpoint was tested to evaluate whether basic access control mechanisms were enforced. Upon sending a simple GET request, the API returned the complete list of posts without requiring any authentication token, API key, or session verification. This indicates that the endpoint is publicly accessible to any anonymous user on the internet.

From a security standpoint, the unrestricted exposure of large datasets introduces several concerns. Attackers could easily automate requests to scrape content, collect information at scale, or abuse the endpoint for reconnaissance. In a real production environment, this behavior may lead to unauthorized data harvesting, increased infrastructure costs, or service degradation.

Postman Evidence:

https://jsonplaceholder.typicode.com/posts Save Share

GET https://jsonplaceholder.typicode.com/posts Send

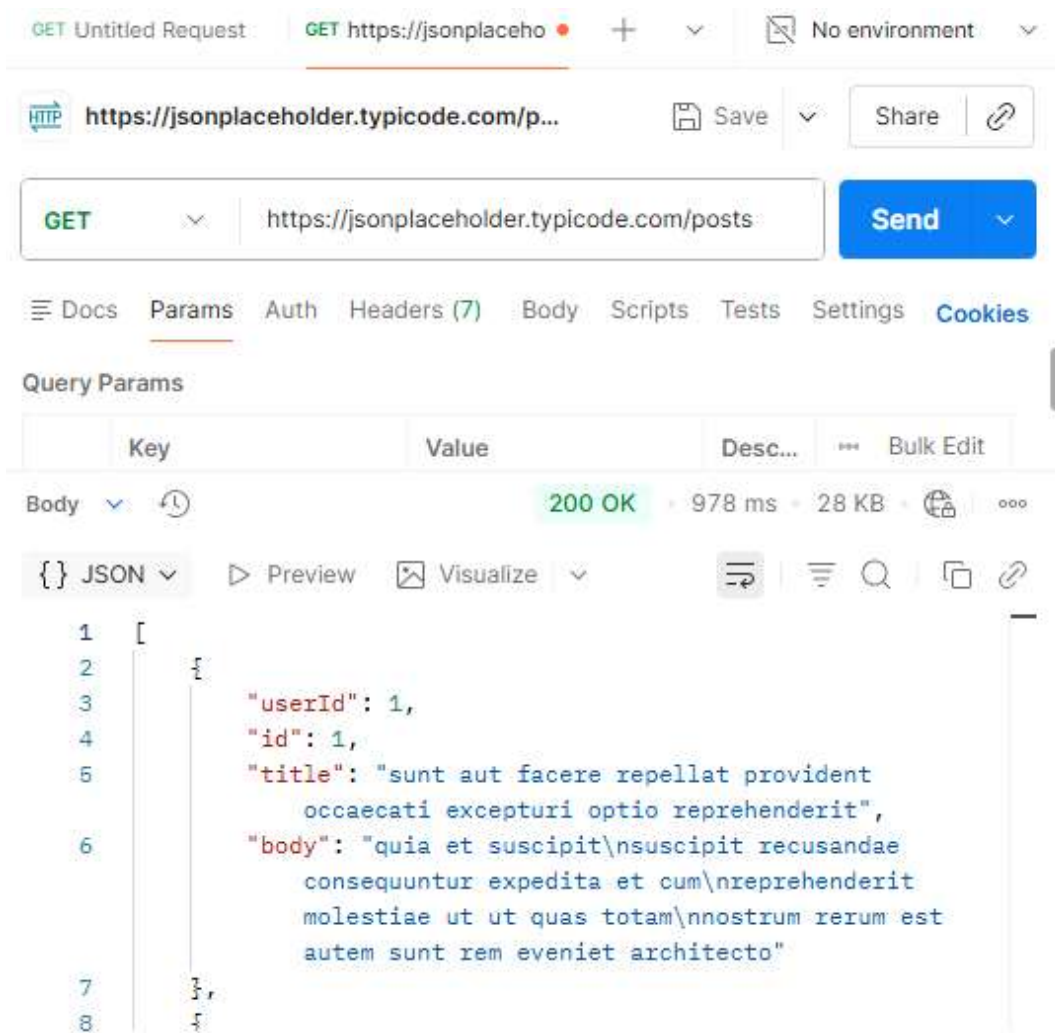
Docs Params Authorization **Headers (7)** Body Scripts Tests Settings Cookies

Headers Hide auto-generated headers

	Key	Value	Descripti...	Bulk Edit	Presets
☒	Cache-Control	no-cache			
☒	Postman-Token	<calculated when request is sent>			
☒	Host	<calculated when request is sent>			
☒	User-Agent	PostmanRuntime/7.51.1			
☒	Accept	*/*			
☒	Accept-Encoding	gzip, deflate, br			
☒	Connection	keep-alive			

Body Cookies **Headers (25)** Test Results 200 OK 978 ms 28 KB ***

Key	Value
Date	Fri, 06 Feb 2026 13:54:52 GMT
Content-Type	application/json; charset=utf-8
Transfer-Encoding	chunked
Connection	keep-alive
access-control-allow-credentials	true



Endpoint Analysis – /comments

The /comments endpoint was assessed to determine whether user-generated data is properly protected. Testing showed that the API exposes all comment records together with associated email addresses. No filtering or authorization checks were applied.

Exposure of email addresses constitutes excessive data disclosure and may facilitate spam campaigns, phishing attacks, or privacy violations. Such information should be accessible only to authorized parties with a legitimate business need.

Postman Evidence:

https://jsonplaceholder.typicode.com/comments

GET https://jsonplaceholder.typicode.com/comments Send

Docs Params Authorization Headers (7) Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (25) Test Results 200 OK 759 ms 155.17 KB

{ } JSON Preview Visualize

```

1 [
2   {
3     "postId": 1,
4     "id": 1,
5     "name": "id labore ex et quam laborum",
6     "email": "Eliseo@gardner.biz",
7     "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
8   },
9   {
10    "postId": 1,
11    "id": 2,
12    "name": "quo vero reiciendis velit similique earum",
13    "email": "Jayne_Kuhic@sydney.com",
14    "body": "est natus enim nihil est dolore omnis voluptatem numquam\net omnis occaecati quod ullam at\nvoluptatem error expedita pariatur\nnihil sint nostrum voluptatem reiciendis et"
15  },
16  {
17    "postId": 1,
18    "id": 3,
19    "name": "odio adipisci rerum aut animi",
20    "email": "Nikita@garfield.biz",
21    "body": "facilis est etiam molestiae eum\nest asperiores sint magnam fuga qui\nautem enim voluptate numquam\nomnis dicitur qui minima est\na voluptatem sed quos quis molestiae\nenim qui voluptatem officia nihil"
22  }
23 ]

```

https://jsonplaceholder.typicode.com/comments

GET https://jsonplaceholder.typicode.com/comments Send

Docs Params Authorization Headers (7) Body Scripts Tests Settings Cookies

Headers Hide auto-generated headers

Key	Value	Description
☒ Cache-Control	no-cache	
☒ Postman-Token	<calculated when request is sent>	
☒ Host	<calculated when request is sent>	
☒ User-Agent	PostmanRuntime/7.51.1	
☒ Accept	*/*	
☒ Accept-Encoding	gzip, deflate, br	
☒ Connection	keep-alive	

Body Cookies Headers (25) Test Results 200 OK 759 ms 155.17 KB

Key	Value
Date	Fri, 06 Feb 2026 14:35:21 GMT
Content-Type	application/json; charset=utf-8
Transfer-Encoding	chunked
Connection	keep-alive
access-control-allow-credentials	true
Cache-Control	max-age=43200

Risk Severity: HIGH

Recommendation: Limit returned fields, apply authentication controls, and ensure personal information is not publicly accessible.

Endpoint Analysis – /users

The /users endpoint was examined to understand how personal profile information is handled. The API returned complete user records, including names, email addresses, and physical address details, again without any authentication requirement.

In a real SaaS environment, this represents a serious privacy and compliance concern. Public exposure of personally identifiable information (PII) could violate data protection regulations and damage user trust. Attackers may leverage such data for social engineering or targeted attacks.

Postman Evidence:

The screenshot shows a Postman interface for a GET request to `https://jsonplaceholder.typicode.com/users`. The request is successful, returning a 200 OK status with a response time of 637 ms and a body size of 2.94 KB. The response body is displayed in JSON format, showing a single user record.

Query Params

Key	Value	Description
Key	Value	Description

Body

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
  }
}
```


https://jsonplaceholder.typicode.com/users

GET https://jsonplaceholder.typicode.com/users Send

Docs Params Authorization Headers (7) Body Scripts Tests Settings Cookies

Headers Hide auto-generated headers

Key	Value	Description
☒ Cache-Control	no-cache	
☒ Postman-Token	<calculated when request is sent>	
☒ Host	<calculated when request is sent>	
☒ User-Agent	PostmanRuntime/7.5.11	
☒ Accept	*/*	
☒ Accept-Encoding	gzip, deflate, br	
☒ Connection	keep-alive	
Key	Value	Description

Body Cookies Headers (26) Test Results 200 OK 637 ms 2.94 KB

Key	Value
Date	Fri, 06 Feb 2026 14:35:39 GMT
Content-Type	application/json; charset=utf-8
Content-Length	1847
Connection	keep-alive
access-control-allow-credentials	true
Cache-Control	max-age=43200

Risk Severity: HIGH

Recommendation: Enforce authentication and authorization checks, apply data minimization, and return only essential fields.

Browser Developer Tools were also used to inspect request headers, response headers, and network timing characteristics. The inspection revealed technology disclosure through headers and demonstrated that responses are delivered without protective restrictions.

Browser DevTools Evidence:

▼ Request Headers

:authority	jsonplaceholder.typicode.com
:method	GET
:path	/posts
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.9
Cache-Control	max-age=0
Cookie	_ga=GA1.1.1538958812.1770321050; _ga_E3C3GCQVBN=GS2.1.s1770392722\$o3\$g1\$1770392737\$j45\$I0\$h0
If-None-Match	W/"6b80-Ybsq/K6GwwqYkAsFqDXGC7DoM"
Priority	u=0, i
Referer	https://jsonplaceholder.typicode.com/
Sec-Ch-Ua	"Not(A:Brand";v="8", "Chromium";v="144", "Google Chrome";v="144"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"Windows"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	same-origin
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36

Activ
Go to !

▼ Response Headers

Access-Control-Allow-Credentials	true
Age	11082
Alt-Svc	h3="443";ma=86400
Cache-Control	max-age=43200
Cf-Cache-Status	HIT
Cf-Ray	9c9bc75469e80015-LHR
Date	Fri, 06 Feb 2026 15:46:08 GMT
Etag	W/"6b80-Ybsq/K6GwwqYkAsFqDXGC7DoM"
Expires	-1
Nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
Pragma	no-cache
Priority	u=0,j
Report-To	[{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=Usoh5gWkiYPGgbhTve5jQedUslzA7NvdNaDkNLzKu94%3D%u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d%u0026ts=1766396843"}],"max_age":3600}]
Reporting-Endpoints	heroku-nel="https://nel.heroku.com/reports?s=Usoh5gWkiYPGgbhTve5jQedUslzA7NvdNaDkNLzKu94%3D%u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d%u0026ts=1766396843"
Server	cloudflare
Server-Timing	cfCacheStatus;desc="HIT"
Server-Timing	cfEdgeDur=5,cfOriginDur=0
Server-Timing	cfExtPri
Vary	Origin, Accept-Encoding
Via	2.0 heroku-router
X-Content-Type-Options	nosniff
X-Powered-By	Express
X-Ratelimit-Limit	1000

Activate Windows
Go to Settings to activate Windows.

```

X Headers Preview Response Initiator Timing Cookies
1 {
2   {
3     "userId": 1,
4     "id": 1,
5     "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6     "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
7   },
8   {
9     "userId": 1,
10    "id": 2,
11    "title": "qui est esse",
12    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non deb
13  },
14  {
15    "userId": 1,
16    "id": 3,
17    "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
18    "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut"
19  },
20  {
21    "userId": 1,
22    "id": 4,
23    "title": "eum et est occaecati",
24    "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam iure\nquis sunt voluptate
25  },
26  {
27    "userId": 1,
28    "id": 5,
29    "title": "nesciunt quas odio",
30    "body": "repudiandae veniam quaerat sunt sed\nnihil aut fugiat sit autem sed est\nvoluptatem omnis possimus esse voluptatibus quis\nnest aut tenetur dolor neque"
31  },
32  {
33    "userId": 1,

```



Risk Severity: HIGH

Recommendation: Implement strong authentication (JWT/OAuth2/API keys), introduce pagination, and apply rate limiting to prevent scraping and abuse.

Consolidated Findings

The assessment identified several recurring weaknesses across all tested endpoints. These include missing authentication mechanisms, excessive data exposure, lack of rate limiting, absence of pagination controls, and disclosure of backend technologies through headers.

Collectively, these issues significantly increase the attack surface and could enable automated abuse or unauthorized access if present in production systems.

Remediation Recommendations

To improve security posture, it is strongly recommended to implement authentication and authorization controls, introduce request throttling, apply pagination to large datasets, minimize response content, and remove unnecessary informational headers. Regular API security testing and adherence to OWASP guidelines should also be adopted as part of the development lifecycle.

Conclusion

This exercise demonstrates practical API security assessment techniques commonly used by security consultants and application security teams. Although the tested platform is intentionally open for learning purposes, similar weaknesses in real systems could result in substantial business risk. Implementing the recommended controls will significantly strengthen resilience against unauthorized access and abuse.